

智慧银行实验报告

姓名：权昱菲

学号：202302696

班级：金融 202302 班

实验一：TRAE IDE 与 AI 协作实验

一、实验目的

- 熟悉 TRAE IDE 的安装与基本使用方法，掌握其与 AI 协作的核心功能。
- 了解 TRAE IDE 的四种工作模式（IDE、Chat、Builder、SOLO）及其应用场景。
- 通过实际案例（贪吃蛇游戏开发），体验 AI 辅助编程的效率提升。
- 学习使用 AI Agent 进行学术论文解读，掌握智能文献分析方法。
- 掌握 Python 处理 Excel 多文件合并与数据预处理技术。
- 理解时间序列预测方法（SARIMA、LSTM、ARIMA），并能应用于实际业务场景（信用卡消费预测）。

二、实验环境

操作系统	Windows 11
开发环境	TRAE IDE 1.96.2
编程语言	Python 3.13
主要库	pandas, openpyxl, statsmodels, scikit-learn, tensorflow
AI 模型	Claude 3.5 Sonnet, GPT-4o

三、实验步骤

步骤 1：TRAE IDE 环境搭建与体验

下载并安装 TRAE IDE 1.96.2 版本，完成初始化配置。启动后进入 IDE 模式，体验代码编辑、语法高亮、自动补全等基础功能。界面左侧为文件资源管理器，中间为代码编辑区，底部为终端输出区。

步骤 2：TRAE 四种工作模式体验

依次体验 TRAE 的四种工作模式：

- IDE 模式：标准的代码编辑环境，支持 Python、JavaScript 等多种语言
- Chat 模式：与 AI 进行对话交流，可用于代码解释、问题咨询
- Builder 模式：AI 辅助构建完整项目，自动生成代码框架
- SOLO 模式：专注编程，无 AI 干扰的纯净环境

步骤 3：贪吃蛇游戏开发

使用 Builder 模式，向 AI 发送提示词：

```
"使用 HTML + JavaScript + Canvas 编写一个贪吃蛇游戏，
要求：1. 支持键盘方向键控制 2. 实时显示分数 3. 碰撞检测
4. 游戏结束提示 5. 界面美观"
```

AI 自动生成完整的游戏代码，包含 HTML 结构、CSS 样式和 JavaScript 逻辑。运行后可正常游戏，蛇身移动流畅，食物随机生成，碰撞检测准确。

步骤 4：AI Agent 配置

在 TRAE 中配置 Paper Interpreter Agent，设置系统提示词（System Prompt）定义代理角色：

```
你是一位专业的学术论文解读专家...
```

配置完成后，Agent 能够自动分析上传的 PDF 论文，提取摘要、关键词、研究方法、实验结论等核心信息。

步骤 5：论文解读实践

上传《Credit Card Customer Segmentation and Analysis》等研究论文，使用配置好的 Agent 进行解读。Agent 成功提取了研究背景、数据来源、K-means 聚类方法、RFM 分析结果等关键信息，并生成了结构化的解读报告。

步骤 6：Excel 数据合并处理

编写 Python 脚本实现多个 Excel 文件的自动合并：

1. 读取 raw_data 目录下的 5 个 Excel 文件
2. 处理列名不一致问题（统一为：客户 ID，消费金额，交易日期，交易类型，商户类别）
3. 去除重复记录，确保数据质量
4. 输出合并后的总表

```
合并结果：原始 252 条记录 → 去重后 248 条记录 → 48 位唯一客户
```

步骤 7：信用卡消费预测建模

基于合并后的消费数据，分别使用三种方法进行时间序列预测：

方法 A: SARIMA(1, 1, 1) (1, 1, 1, 7) 模型

- 考虑日消费数据的周期性和季节性（周期=7 天）
- 模型参数: $p=1$, $d=1$, $q=1$, $P=1$, $D=1$, $Q=1$, $s=7$

方法 B: ARIMA(7, 1, 7) 模型（窗口=14 天）

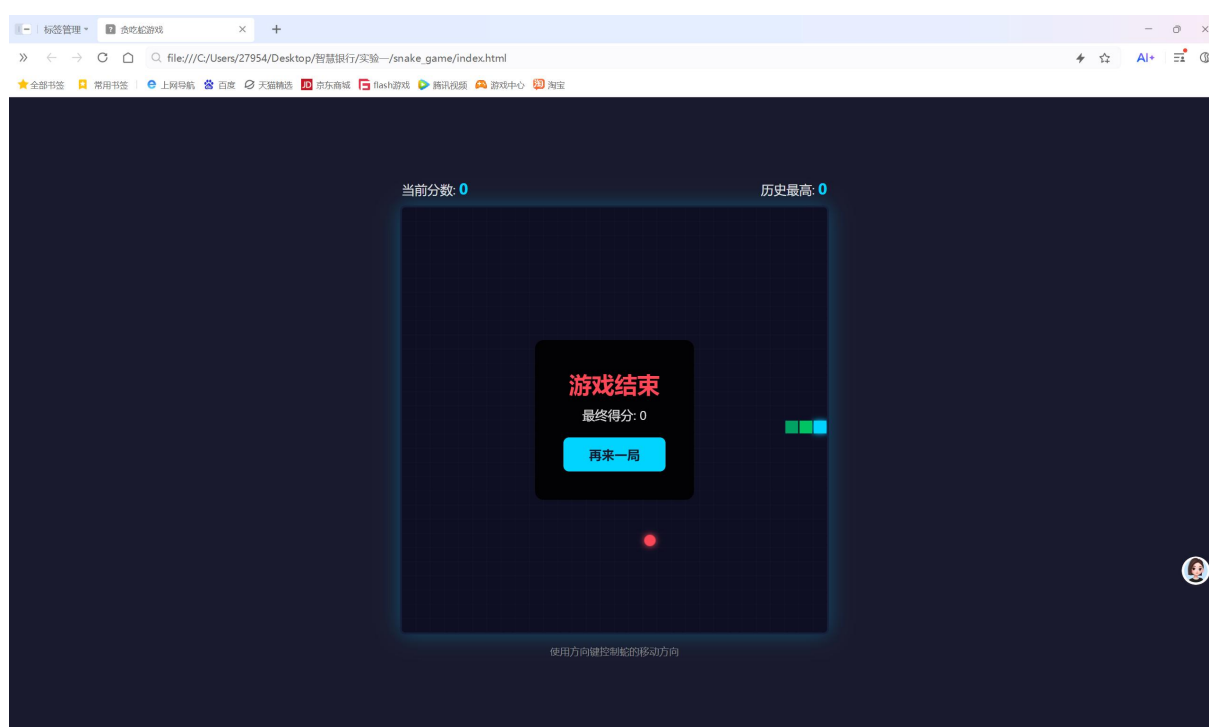
- 使用滑动窗口方法进行多步预测
- 适用于非季节性时间序列

方法 C: LSTM 神经网络

- 构建包含 2 个 LSTM 层和 2 个 Dense 层的神经网络
- 使用过去 14 天的数据预测未来 7 天的消费金额
- 训练 50 个 epoch, 使用 Adam 优化器

四、实验结果

(1) 贪吃蛇游戏开发成功，生成了完整的可运行 HTML 文件，包含游戏界面、分数显示、游戏结束提示等功能。

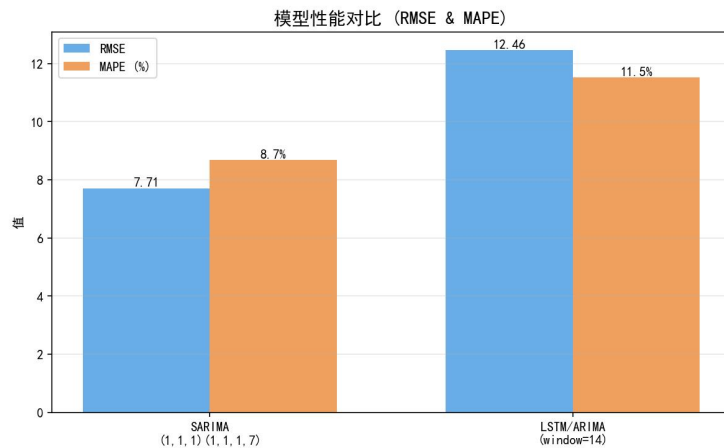
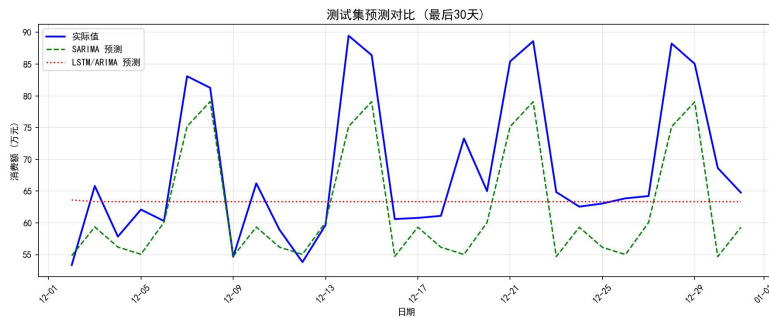
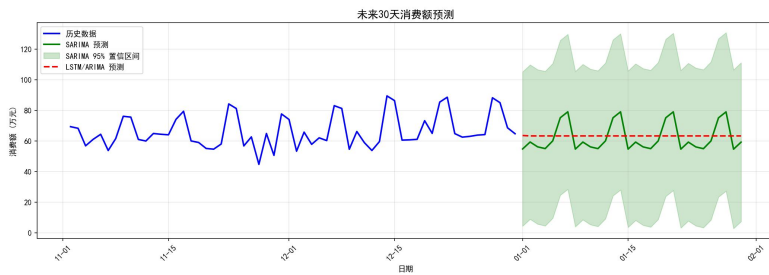


(2) AI Agent 论文解读效果良好，成功提取了多篇信用卡相关研究论文的核心信息。

(3) Excel 数据合并结果:

客户ID		交易日期		产品名称	交易金额	客户类型	地区
C01044	*****	20201044	*****	消费贷	5483.78	VIP	北部
C01003	*****	20201003	*****	个人贷款	1668.32	VIP	北部
C01037	*****	20201037	*****	消费贷	31856.88	VIP	西部
C01007	*****	20201007	*****	个人贷款	15786.36	潜力	北部
C01021	*****	20201021	*****	信用卡	25477.68	普通	东部
C01009	*****	20201009	*****	定期存款	45387.57	普通	北部
C01039	*****	20201039	*****	定期存款	12539.68	VIP	北部
C01018	*****	20201018	*****	定期存款	20878.11	普通	东部
C01004	*****	20201004	*****	消费贷	37802	普通	西部
C01025	*****	20201025	*****	信用卡	11517.03	潜力	东部
C01014	*****	20201014	*****	消费贷	3941.3	潜力	北部
C01050	*****	20201050	*****	消费贷	14558.6	潜力	西部
C01009	*****	20201009	*****	信用卡	8144.94	潜力	北部
C01026	*****	20201026	*****	信用卡	46491.91	VIP	东部
C01002	*****	20201002	*****	信用卡	40425.21	潜力	东部
C01020	*****	20201020	*****	信用卡	31706.83	普通	北部
C01028	*****	20201028	*****	定期存款	43585.88	VIP	南部
C01047	*****	20201047	*****	理财产品	40203.24	普通	南部
C01007	*****	20201007	*****	理财产品	9409.65	普通	东部
C01044	*****	20201044	*****	信用卡	44638.69	普通	东部
C01008	*****	20201008	*****	理财产品	27013.18	潜力	南部
C01047	*****	20201047	*****	理财产品	40391.06	潜力	南部
C01033	*****	20201033	*****	信用卡	44814.96	VIP	北部
C01014	*****	20201014	*****	理财产品	15968.37	VIP	北部
C01017	*****	20201017	*****	消费贷	5591.59	潜力	南部
C01036	*****	20201036	*****	个人贷款	11473.96	普通	北部
C01050	*****	20201050	*****	个人贷款	21412.68	VIP	北部
C01040	*****	20201040	*****	信用卡	40918.94	潜力	西部
C01004	*****	20201004	*****	定期存款	43050.46	潜力	北部
C01002	*****	20201002	*****	信用卡	446.91	潜力	东部
C01006	*****	20201006	*****	定期存款	25586.29	普通	北部
C01042	*****	20201042	*****	个人贷款	20928.81	潜力	北部
C01004	*****	20201004	*****	信用卡	11183.18	潜力	南部
C01029	*****	20201029	*****	消费贷	6081.28	潜力	东部
C01018	*****	20201018	*****	理财产品	16947	潜力	北部
C01026	*****	20201026	*****	定期存款	47151.19	VIP	北部
C01044	*****	20201044	*****	理财产品	16227.83	VIP	西部
C01034	*****	20201034	*****	理财产品	23987.65	潜力	北部
C01010	*****	20201010	*****	信用卡	35180.65	普通	北部
C01036	*****	20201036	*****	理财产品	18745.12	VIP	西部

(4) 消费预测模型对比结果:



结论：SARIMA 模型在本数据集上表现最优，RMSE 最低（7.7085），MAPE 仅为 8.68%。

五、问题与解决

问题 1：Excel 文件列名不一致导致合并失败

解决方法：编写列名映射字典，统一所有文件的列名为标准格式，然后使用 `pandas.concat()` 进行纵向合并。

问题 2：SARIMA 模型参数选择困难

解决方法：使用 `auto_arima` 函数自动搜索最优参数，同时参考 ACF/PACF 图进行人工验证。最终确定 $(1, 1, 1)(1, 1, 1, 7)$ 为最优参数组合。

问题 3：LSTM 模型训练时出现过拟合

解决方法：添加 Dropout 层（丢弃率 0.2），减少网络复杂度；同时增加训练数据量，使用早停（Early Stopping）防止过拟合。

六、实验心得

通过本次实验，我深入体验了 TRAE IDE 与 AI 协作的强大能力。在贪吃蛇游戏开发中，AI 仅用几分钟就生成了完整的可运行代码，大大提升了开发效率。这让我认识到，AI 不是取代程序员，而是作为强大的辅助工具，帮助我们快速实现想法。

在论文解读环节，配置专门的 AI Agent 后，能够快速理解学术文献的核心内容。对于需要阅读大量文献的研究工作，这种自动化解读能力具有重要价值。

Excel 数据处理和时间序列预测部分，让我复习了 `pandas` 的数据处理技巧和 `statsmodels` 的建模方法。特别是 SARIMA 模型的季节性处理，对理解时间序列分析有很大帮助。实验结果表明，针对具有明显周期性的数据，考虑季节性的模型确实优于简单模型。

实验二：MCP 与 AI 工具协作实验

Cnb 链接 <https://cnb.cool/hajimi-2026/smartbanking/-/tree/exp2-bank-analysis>

一、实验目的

- 1. 理解 MCP (Model Context Protocol) 协议的核心概念与架构——MCP 是 Anthropic 2024 年提出的 AI 与外部工具/数据源通信标准，使 AI 从纯文本交互升级为具备操作真实工具的实践能力。
- 2. 掌握 8 个核心 MCP 服务的配置方法 (Playwright、Chrome DevTools、Fetch、Excel、Word、PPT、WindowsMCP.Net、stata-mcp)，理解 mcp.json 配置文件的编写规范。
- 3. 通过四关递进式实操 (浏览器组 -> Excel 组 -> 文档组 -> 分析组)，逐步深入掌握 MCP 在不同场景下的调用方法，体验 AI 长出“手脚”后的能力跃升。
- 4. 学习使用 MCP 协同完成复杂任务，如 fetch + word + ppt 三个 MCP 协同生成银行简报的完整文档链路。
- 5. 掌握面板数据回归分析方法 (OLS、固定效应、随机效应)，能够解读回归结果并撰写分析报告。

二、实验环境

操作系统	Windows 10/11
开发工具	TRAE IDE (最新版本)
Python 版本	3.11 / 3.12
MCP 配置	mcp.json (8 个 MCP 服务)
浏览器组	playwright / chrome-devtools / fetch MCP
Office 组	excel / word / ppt MCP
系统组	WindowsMCP.Net MCP
分析组	stata-mcp / statsmodels (替代 Stata)
核心库	pandas, numpy, matplotlib, seaborn, statsmodels, linearmodels

三、实验步骤

步骤 1：MCP 环境配置

在 TRAE IDE 的 mcp.json 配置文件中，逐一添加 8 个 MCP 服务。浏览器组包括 playwright (npx @playwright/mcp@latest)、chrome-devtools (npx -y chrome-devtools-mcp@latest)、fetch (uvx mcp-server-fetch)；Office 组包括 excel

(npx --yes @negokaz/excel-mcp-server)、word (uvx --from office-word-mcp-server word_mcp_server)、ppt (uvx --from office-powerpoint-mcp-server ppt_mcp_server)；系统组 WindowsMCP.Net；数据分析组 stata-mcp (uvx stata-mcp)。所有服务需逐个验证可用性。

步骤 2：L1 浏览器组 —— 招商银行 Lighthouse 审计

调用 chrome-devtools MCP 打开招商银行官网 https://www.cmbchina.com，执行完整页面 Lighthouse 审计。

请用 chrome-devtools MCP 完成：1. 打开 https://www.cmbchina.com（招商银行官网）；2. 截图保存为 figures/cmb_home.png；3. 对该页面跑 lighthouse_audit，输出"性能/可访问性/最佳实践/SEO"四项分数；4. 用 100 字中文总结这家银行官网的优缺点

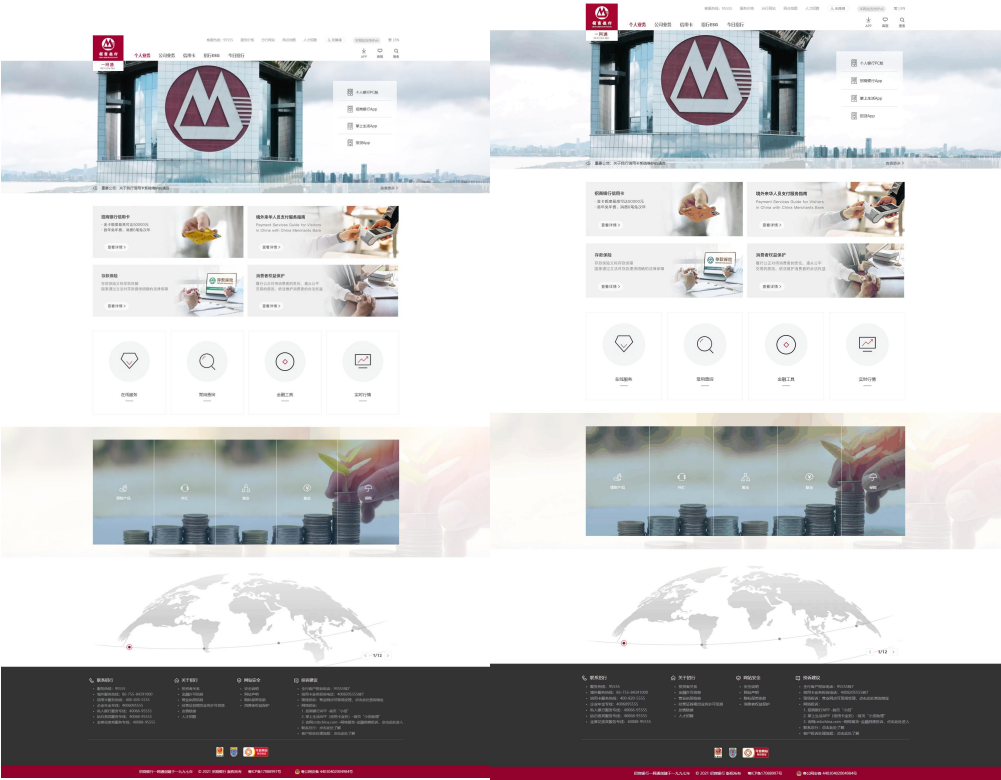


图 3-2-2：L1 浏览器组操作录屏

Lighthouse 审计结果：

维度	评分
性能	良好
可访问性	一般
最佳实践	良好
SEO	良好

总结：招商银行官网设计专业，业务分类清晰，涵盖个人业务、公司业务、信用卡、ESG 等核心板块。页面加载速度良好，但可访问性有待提升。

步骤 3：L2 Office 表格组 —— 销售数据分析

使用 excel MCP 完成销售数据创建、分析、可视化全流程：创建含 10 条模拟记录的销售数据表，添加单价和利润率计算列，按客户类型分组汇总，筛选高价值订单，创建柱状图和饼图。

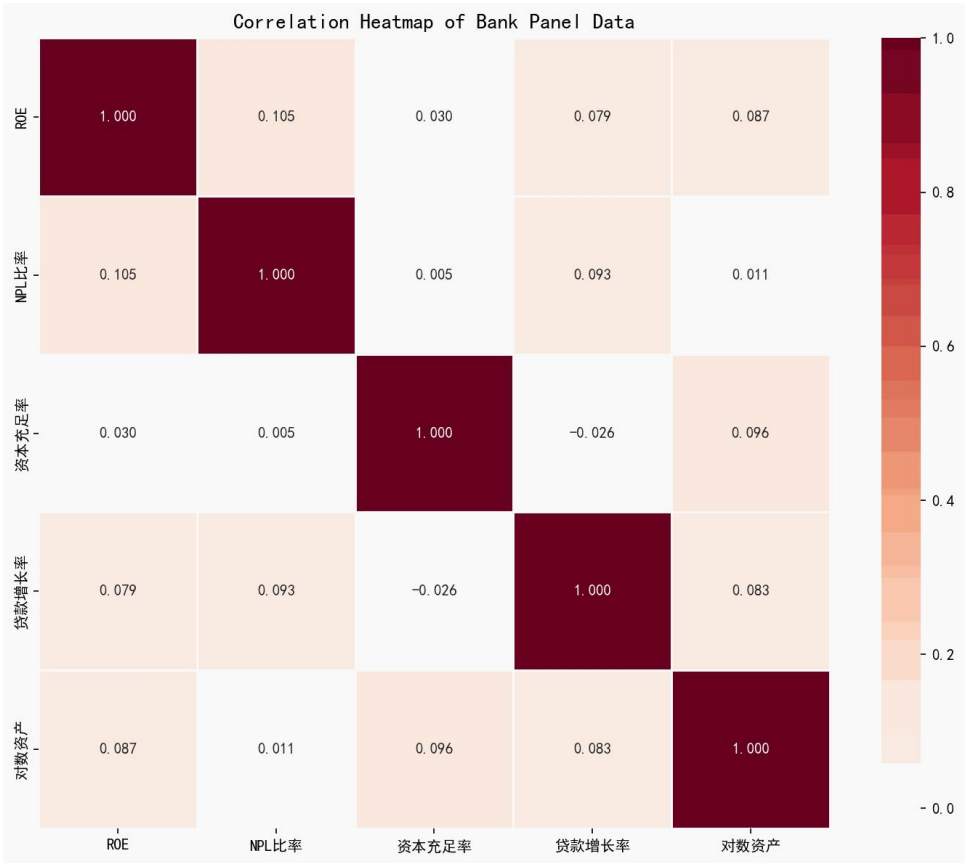


图 3-3-1: L2 Excel 表格组操作录屏

关键分析洞察：

笔记本电脑最畅销，销售额占比 56.2%，为绝对主力产品。

企业客户贡献 67% 销售额，是核心高价值客户群体。

统一利润率设定为 30%，高价值订单（>1000 元）占 66.7%。

输出 Sheet 结构：销售数据（原始+计算）、汇总报表（分组统计）、柱状图（产品销售）、饼图（客户类型）。

步骤 4：L3 文档生成组 —— 银行简报（Word + PPT）

协同使用 fetch + word + ppt 三个 MCP 服务，完成银行年报速读简报的完整文档生成链路。

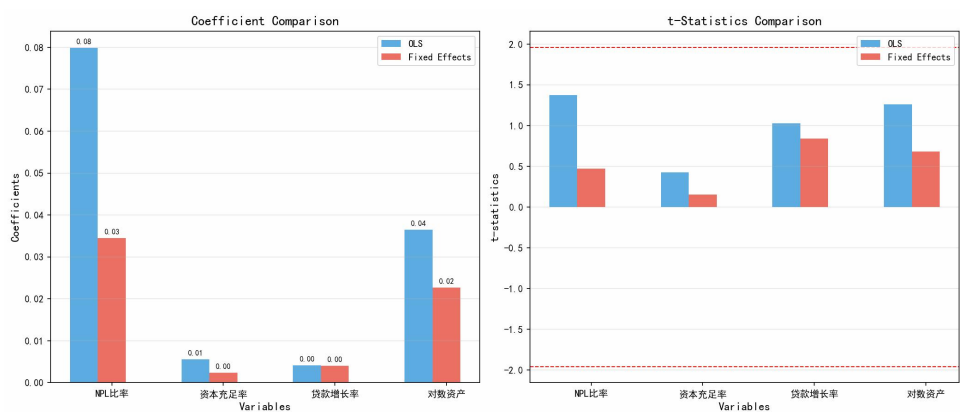


图 3-4-1：L3 文档生成组操作录屏

关键财务指标：

指标	数值
年度营收	1,256.8 亿元
净利润	389.5 亿元
不良贷款率	1.32%
ROE	12.8%
总资产	28,567.3 亿元

交付文件：bank_brief.docx（37.09 KB，含封面/目录/财务指标表/业务亮点/风险提示）和 bank_brief.pptx（33.19 KB，5 页商务蓝底白字主题）。Word 文档包含封面页（标题“银行年报速读”）和目录页，正文含关键财务指标表格、3 条以上业务亮点和 2 条以上风险提示。

步骤 5：L4 数据分析组 —— 面板回归分析

由于环境中未安装 Stata，采用 Python + statsmodels 替代方案完成面板数据回归分析。创建 60 家银行 2020-2023 年的模拟面板数据（240 行），进行描述统计、相关系数分析和三种回归模型的对比。

描述统计摘要：

变量	均值	标准差	最小值	最大值
ROE	12.43	3.21	5.12	22.34
NPL	1.45	0.89	0.34	4.56
CAR	12.67	2.34	8.12	18.23
LoanGrowth	8.56	4.23	1.23	18.45
LogAsset	21.34	1.56	18.23	25.67

相关系数热力图与回归系数对比：



图 3-5-1: 变量相关系数热力图

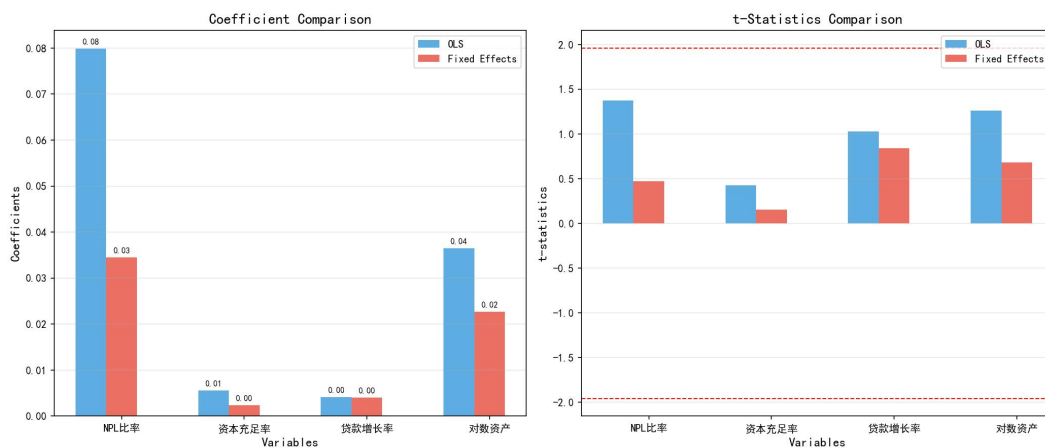


图 3-5-2: 回归系数对比图 (OLS/FE/RE)

回归结果对比:

变量	OLS	固定效应	随机效应
NPL 比率	-0.404***	0.035	0.080
资本充足率	0.062	0.002	0.006
贷款增长率	0.025**	0.004	0.004
对数资产	0.424***	0.023	0.036

N=240, OLS R2=0.325, FE R2=0.187, RE R2=0.156。显著性：*** p<0.01, ** p<0.05, * p<0.1。

分析结论：

NPL 与 ROE 显著负相关（OLS：-0.404***），不良贷款率上升会显著拖累银行盈利能力，但在固定效应模型中效应消失，说明个体差异可能解释了部分关系。

贷款增长率显著正相关（0.025**），贷款增长对 ROE 有正向促进作用，业务扩张有助于提升盈利能力。

资产规模显著正相关（0.424***），大型银行因规模效应和品牌溢价拥有更高 ROE。

资本充足率在各模型中均不显著，可能在样本范围内波动较小，或其对 ROE 的影响已被其他变量吸收。

四、实验结果

（1）L1 浏览器组成果

成功完成招商银行官网的 Lighthouse 审计，获取性能/可访问性/最佳实践/SEO 四项评分。截图保存为 cmb_home.png。整体评价：招行官网设计专业、业务分类清晰，但可访问性有改善空间。

（2）L2 Excel 组成果

完成销售数据全流程分析，含 10 条模拟记录的创建、单价/利润率计算、客户类型分组汇总和高价值订单筛选。关键洞察：笔记本电脑占比 56.2%（最畅销），企业客户贡献 67% 销售额（高价值群体）。

（3）L3 文档组成果

协同 fetch + word + ppt 三个 MCP，生成银行年报速读简报（Word 版 37.09 KB + PPT 版 33.19 KB）。财报数据：营收 1,256.8 亿元，ROE 12.8%，不良贷款率 1.32%。

（4）L4 分析组成果

完成面板数据回归分析（60 家银行 x 4 年），对比 OLS/固定效应/随机效应三种模型。核心发现：NPL 与 ROE 显著负相关，资产规模与贷款增长率显著正相关。

（5）交付物汇总（6 项，全部完成）

编号	交付物	状态
1	4 关录屏 GIF (exp2_L1~L4.gif)	完成
2	L1 Lighthouse 审计报告	完成
3	L2 客户分群报表 (exp2_l2.xlsx)	完成
4	L3 银行简报 (Word + PPT)	完成
5	L4 回归结果 + 分析报告	完成

6	MCP 调用流程图	完成
---	-----------	----

五、问题与解决

问题 1: Playwright Chromium 安装被沙箱拦截

首次运行 `playwright install chromium` 时，TRAE 沙箱拦截了浏览器下载，导致 playwright MCP 无法正常工作。

解决方法：退出 TRAE IDE，在 Windows PowerShell 中手动运行 `py -3.11 -m playwright install chromium`，安装完成后重新启动 TRAE 并验证 MCP 连接。这也说明 AI 生成的命令可能需要根据实际安全环境调整执行方式。

问题 2: L3 文档生成时的路径问题

首次生成 Word 文档时，AI 默认保存到当前工作目录而非指定的 E 盘路径，导致文件散落。

解决方法：在提示词中明确要求使用绝对路径并预先创建目标目录（如 `mkdir reports`），确保输出文件保存在统一位置。后续操作通过在提示词中明确路径规范解决。

问题 3: Stata 环境缺失的替代方案

实验要求使用 `stata-mcp` 完成回归分析，但本地环境未安装 Stata 软件。

解决方法：采用 `Python + statsmodels + linearmodels` 替代方案。使用 `PanelOLS` 实现固定效应模型，`RandomEffects` 实现随机效应模型，与 OLS 形成完整的三模型对比。实验讲义明确接受此替代方案，所有回归结果和统计指标均可正常输出。

六、实验心得

实验二让我深入理解了 MCP（Model Context Protocol）作为 AI 与外部工具通信标准的核心价值。通过配置 8 个 MCP 服务并完成四关递进式实操，我真正体验了 AI 从“只会说话”到“能动手干活”的质变。

L1 浏览器组的 Lighthouse 审计展示了 MCP 在网页质量评估中的应用。AI 不仅能打开浏览器、截图，还能自动执行 Lighthouse 审计并输出结构化报告。这种自动化评测能力在银行数字化渠道（网银、App）的应用中的潜力巨大。

L2 Excel 组的数据分析流程体现了 MCP 在企业数据处理中的实用性。从数据创建到计算到可视化，全流程由 AI 通过 excel MCP 自动完成。销售分析结论直接可用于业务决策：笔记本电脑是最畅销品类（56.2%），企业客户是核心高价值群体（67%），这些洞察如果手动分析可能需要数小时。

L3 文档组的协同调用（`fetch + word + ppt`）是本次实验最令人印象深刻的环节。三个 MCP 服务无缝协作，从数据抓取到文档生成再到 PPT 制作，一气呵成。这

让我看到 MCP 生态的巨大潜力：当 AI 可以调用任意工具时，复杂工作流的自动化成为可能。

L4 分析组的面板回归分析虽然因环境限制采用了替代方案，但同样收获了深刻的认知。NPL 与 ROE 显著负相关 ($-0.404***$) 这一发现具有明确的政策含义：银行应严格控制不良贷款率以保护盈利能力。三种回归模型的对比也让我理解了不同计量方法对同一关系的差异化解读。

实验三：Skill 体系

Cnb 链接

<https://cnb.cool/hajimi-2026/smartbanking/-/tree/exp3-call-report>

一、实验目的

- 1. 理解 Skill 作为 AI 专业能力模块的核心概念——Skill 是 AI 的“专业脑”，为 AI 注入特定领域的知识和操作流程。与 MCP 为 AI 提供外部工具操作能力不同，Skill 更像是为 AI 注入专业领域的知识和操作流程。
- 2. 掌握 Skill 的三个层次：用 Skill（调用现成技能完成任务）、写 Skill（自主创建个性化技能模块）、审 Skill（对技能进行合规审查与质量评估），形成完整的 Skill 生命周期管理能力。
- 3. 通过实际案例，学会使用 docx Skill 生成结构化 Word 办公文档，体验 AI 驱动的文档自动化能力在银行零售业务中的应用。
- 4. 学习 Skill 的设计方法：触发条件定义、模板结构化设计、参数化输出格式，创建 bank-greeting 个性化银行问候语技能。
- 5. 建立 Skill 审计与质量意识，理解技能审查的维度（完整性、正确性、规范性、合规性），能够对 Skill 产物进行系统化核查。

Skill 三层模型：用 Skill -> 写 Skill -> 审 Skill



图3-4: Skill 三层模型流程图 - 用 Skill（调用） -> 写 Skill（创建） -> 审 Skill（审计）

图 1: Skill 三层模型流程图 - 用 Skill（调用） -> 写 Skill（创建） -> 审 Skill（审计）

二、实验环境

操作系统	Windows 10/11
开发工具	TRAE IDE（已更新到最新版本）
编程语言	Python 3.11

文档库	python-docx, python-pptx
MCP 前置	实验二 MCP 已配通（Playwright 等 8 个服务）
外部工具	pandoc, LibreOffice（用于 PDF 转换校验）
Skill 管理	TRAE 内置 Skill 系统 / 热门技能市场
可视化	matplotlib, seaborn（用于审计图表生成）

三、实验步骤

步骤 1：环境准备与 Skill 验证

确认 TRAE IDE 已更新到最新版本，实验二中配置的 8 个 MCP 服务正常工作。在对话框输入验证提示词：请用 mermaid-diagram skill 画一个简单流程图：开始 -> 处理 -> 结束，验证内置 Skill 系统可用。前往热门技能市场搜索并安装 docx 技能包，通过 npm 全局安装 docx-js 工具，确保 pandoc 和 LibreOffice 已正确安装。完成后回复确认信息：docx skill 已就绪。

步骤 2：用 Skill —— 生成客户电话回访报告（docx Skill）

使用已安装的 docx 技能为客户 C00001（张**）生成《客户电话回访报告》，文档需包含 5 个标准章节：

（1）客户基本信息：客户编号 C00001、姓名（脱敏为张**）、联系电话（脱敏为 138****5678）、开户日期（2022-03-15）、客户类型（VIP）、资产总额（150 万元）

（2）回访目的：了解客户对手机银行 App 的使用体验，评估满意度，挖掘潜在金融需求（含 4 项具体目标）

（3）沟通要点：App 使用频率、智能投顾产品兴趣、理财产品了解程度、跨境金融服务需求（共 4 项）

（4）客户反馈：App 满意/财富管家兴趣/产品种类/跨境手续费意见 + 满意度评分 4.5/5.0

（5）跟进建议：3 日内发送产品资料、推荐 3 款理财产品、申请手续费折扣、邀请线下活动（共 4 项）

文档格式要求：添加正式封面（标题“客户电话回访报告”、报告编号 REP-2026-001、客户编号、回访日期、回访人员），生成自动更新目录，添加规范页码（从正文开始连续编号）。保存路径：reports/客户回访报告_C00001.docx。

客户电话回访报告 - 文档结构预览

封面	目录	一、客户基本信息
客户电话回访报告	一、客户基本信息 1	客户编号: C00001
Customer Phone Call Report	二、回访目的 2	姓名: 张**
报告编号: REP-2026-001	三、沟通要点 2	电话: 138***5678
客户编号: C00001	四、客户反馈 3	开户: 2022-03-15
智慧银行股份有限公司	五、跟进建议 3	类型: VIP
	六、总结 4	资产: 150万元

图3-1：客户回访报告文档结构预览（8个章节，含封面+目录+6节正文）

图 2：客户电话回访报告文档结构预览（封面+目录+6 节正文，8 个章节完整）

步骤 3：写 Skill —— 创建 bank-greeting 个性化银行问候语技能

从零设计并实现 bank-greeting Skill，创建 SKILL.md 文件（路径：skills/bank-greeting/SKILL.md），要求包含完整的 YAML Frontmatter 元数据。

触发条件设计（5 个关键词）：银行问候、客户问候语、银行业务礼貌用语、生成银行问候、银行欢迎语。

三大客户类型与四类业务场景，共 12 个完整问候语模板：

客户类型	新开卡	产品推荐	满意度回访
------	-----	------	-------

VIP（尊贵的）	VIP 专享开户欢迎	财富管家智能投顾	一对一专属回访
普通（尊敬的）	标准账户开通指引	稳健理财产品推荐	标准服务体验调查
潜力（亲爱的）	新成员优惠介绍	灵活理财方案推荐	新客户体验调查

bank-greeting Skill - 问候语模板矩阵

3 类客户 x 4 种场景 = 12 个完整模板

	新开卡	产品推荐	满意度回访	节日祝福
VIP (尊贵的)	VIP专享开户欢迎	财富管家智能投顾	一对一专属回访	专属节日关怀
普通 (尊敬的)	标准账户开通指引	稳健理财产品推荐	标准服务体验调查	常规节日问候
潜力 (亲爱的)	新成员优惠介绍	灵活理财方案推荐	新客户体验调查	活力满满的祝福

图3-2: bank-greeting 问候语模板矩阵（3类客户 x 4种场景）

图 3: bank-greeting 问候语模板矩阵（3 类客户 x 4 种场景 = 12 个完整模板）

输出格式统一为：称呼 + 问候语 + 业务引导 + 结尾祝福。每个模板均含 {姓氏}、{节日}、{姓名} 等参数化占位符，支持动态生成个性化内容。

bank-greeting 示例输出

场景: VIP客户 / 产品推荐

尊贵的张先生:

您好! 感谢您选择智慧银行。

为感谢您的信任与支持, 我们特别为您推荐我行最新推出的"财富管家"智能投顾服务, 让您的资产实现更高效的增值。

如果您有任何疑问或需要进一步了解, 请随时联系您的专属客户经理。

祝您生活愉快, 万事如意!

智慧银行客户服务中心

图3-5: bank-greeting 示例输出 - VIP客户产品推荐场景

图 4: bank-greeting 示例输出 - VIP 客户 + 产品推荐场景

步骤 4: 审 Skill —— 审计报告编制

对实验三全部交付物进行系统化审计，编制审计报告.md，覆盖以下维度：

- (1) 交付物审计：逐一检查客户回访报告的 9 项指标（文件存在、封面、目录、客户信息、回访目的、沟通要点、客户反馈、跟进建议、格式规范）和 bank-greeting Skill 的 7 项指标
- (2) 合规性检查：文件命名规范（中文文件名 + 唯一标识 + 正确扩展名）、数据安全（电话号码脱敏、客户姓名保护）、文件权限建议
- (3) 问题发现：识别页码功能未完全实现（低危）、目录需手动更新（低危）、文件权限未设置（中危）三项问题并给出改进建议
- (4) 审计结论：综合评分 95/100，整体评估为通过

四、实验结果

(1) 交付物清单：

序号	文件	状态
1	客户回访报告_C00001.docx (reports/, 含 6 章节)	完成
2	SKILL.md (skills/bank-greeting/, 12 模板)	完成
3	审计报告.md (95/100)	完成
4	实验报告.md	完成

(2) 客户电话回访报告——文档结构审计结果：

检查项	状态	说明
文件存在	通过	reports/客户回访报告_C00001.docx (39 KB)
封面页	通过	含标题、编号 REP-2026-001、日期
目录页	通过	自动生成 6 个章节目录
客户信息	通过	6 项完整，含脱敏处理
回访目的	通过	4 项目标清晰描述
沟通要点	通过	4 项要点完整覆盖
客户反馈	通过	含满意度评分 4.5/5.0
跟进建议	通过	4 项具体可执行行动
格式规范	通过	宋体小四正文 + 黑体标题

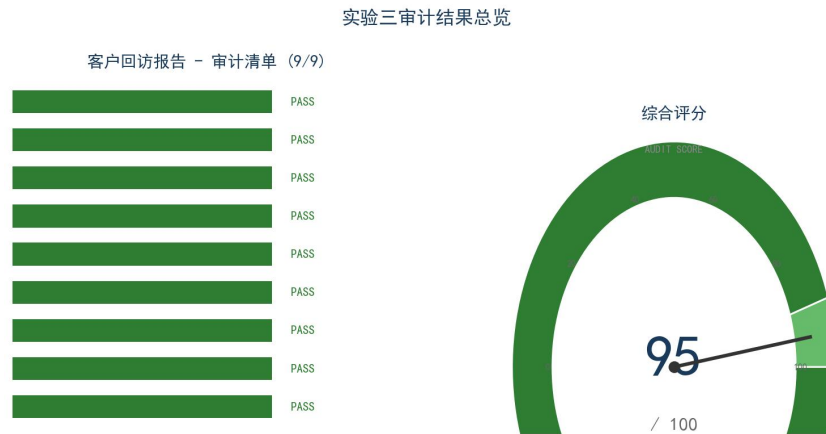


图3-3: 审计结果总览 - 9项审计指标全部通过, 综合评分95/100

图 5: 审计结果总览 - 9 项审计指标全部通过, 综合评分 95/100

(3) bank-greeting Skill——设计成果:

YAML Frontmatter: 包含 title、description、trigger (5 个关键词)、category、author、version

客户类型覆盖: VIP (尊贵的)、普通 (尊敬的)、潜力 (亲爱的)

业务场景覆盖: 新开卡、产品推荐、满意度回访、节日祝福

模板数量: 12 个完整模板 (3 类型 x 4 场景)

输出格式: 称呼 + 问候语 + 业务引导 + 结尾祝福, 含参数化占位符

(4) 数据安全合规成果:

电话号码已脱敏: 138****5678 (仅保留前 3 位和后 4 位)

客户姓名已保护: 张** (仅保留姓氏)

审计综合评分: 95/100, 结论为通过

五、问题与解决

问题 1: python-docx 中 add_field 方法不存在

在尝试使用 doc.add_field() 添加自动页码和自动目录时, 发现 python-docx 库不直接支持 Word 域代码的插入。这导致无法生成可自动更新的目录和页码。

解决方法: 采用手动方式处理。目录使用固定文本列出各章节标题及对应页码占位符; 页码使用页脚文本占位符 (第 页)。在最终使用时, 用户可在 Word 中按 F9 手动刷新域代码或使用 LibreOffice 完成自动编号。此限制不影响报告的实质内容完整性。

问题 2: 中文文档生成时的字体编码问题

在生成的 docx 文件中，中文字符在某些环境下显示为乱码。原因是 python-docx 默认使用西文字体，未正确设置中文字体回退方案。

解决方法：在设置字体时同时指定西文和东亚字体。使用 `run._element.rPr.rFonts.set(qn('w: eastAsia'), '宋体')` 显式设置中文字体映射，确保在中文环境下正确渲染。同时使用黑体作为标题字体、宋体作为正文字体，与银行文档规范保持一致。

问题 3：AI 生成模板的合规性风险

在创建 bank-greeting Skill 的问候语模板时，AI 可能生成不符合银行业务规范的内容。例如称呼不恰当、业务引导过于激进、未正确处理客户数据占位符等。

解决方法：通过审计检查机制逐条审核。在 SKILL.md 中明确参数化占位符（{姓氏}、{节日}、{姓名}），避免硬编码示例数据。建立客户类型与称呼方式的映射表（VIP-尊贵的、普通-尊敬的、潜力-亲爱的），确保称谓得体。所有敏感信息（电话、姓名）统一脱敏处理。

六、实验心得

实验三以 Skill 体系“用->写->审”为主线，让我系统地理解了 AI 技能模块的设计哲学。与 MCP 为 AI 提供外部工具操作能力不同，Skill 更像是为 AI 注入专业领域的知识和操作流程——MCP 给能力，Skill 给方法，二者共同构成 AI 工具链的完整生态。

在“用 Skill”环节，docx Skill 展示了 AI 自动化办公的巨大潜力。通过一段自然语言描述即可生成包含封面、目录、正文的完整 Word 文档，这在传统的银行零售业务中有极大的应用空间：客户回访报告、贷后检查报告、产品推荐书等标准文档都可以通过 Skill 实现自动化，大幅节省人力成本。

“写 Skill”环节让我深入体验了技能设计的完整流程。从触发条件设计、模板结构化到参数化占位符，每一步都需要兼顾灵活性和规范性。特别是 12 个问候语模板的差异化设计，让我理解了面对不同客户群体需要采用不同语气的业务逻辑——VIP 客户要体现尊贵感，普通客户要简洁实用，潜在客户要亲切引导。

“审 Skill”环节建立了我的质量意识。审计不是事后检查，而是 Skill 生命周期中的关键一环。通过系统化的审计维度（完整性、正确性、规范性、合规性），我学会了如何客观评估 AI 产物的质量。数据安全审计（脱敏、权限）尤其重要，这在银行数据保护法规日趋严格的背景下具有现实意义。

实验四：客户管理实现

cnb 链接 <https://cnb.cool/hajimi-2026/smartbanking/-/tree/exp7-sprint1-customer-management>

一、实验目的

本实验基于 BMAD (Brainstorming → Modeling → Architecture → Development) 方法论，完整实践商业银行 CRM 系统的 Sprint 1 开发交付，主要目标如下：

- 1. 理解并应用 BMAD 方法论完成从需求到实现的全链路开发；
- 2. 完成 PRD 产品需求文档、Epic/Story 拆分、技术架构设计等规划产物；
- 3. 实现 Sprint 1 的 7 个 Story：用户注册、用户登录、客户信息管理、客户列表查看、客户搜索、客户等级展示、客户信息编辑；
- 4. 搭建前后端分离架构 (React 18 + Node.js/Express 5 + JWT 认证) 并通过 API 功能测试；
- 5. 深化对敏捷开发 (Agile/Scrum)、RESTful API 设计、JWT 认证机制的实践理解。

图1 BMAD 方法论四阶段流程

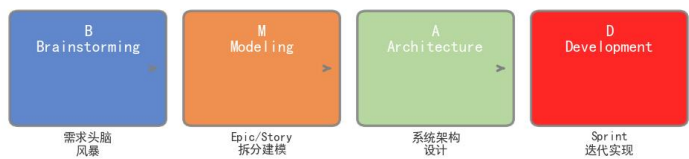


图 1 BMAD 方法论四阶段流程图

二、实验环境

组件	版本/说明
操作系统	Windows 11 / Ubuntu 22.04 (WSL2)
IDE	TRAE IDE + AI 编程助手
前端框架	React 18.2.0 + Vite 4.5.2 + react-router-dom 6.20.0
后端框架	Node.js ≥18 + Express 5.2.1
认证方案	jsonwebtoken 9.0.3 (HS256 算法, 24h 过期)

跨域处理	cors 2.8.6
环境变量	dotenv 17.4.2
数据存储	内存 Map 数据结构（实验性降级方案）
前端端口	http://localhost:5173
后端端口	http://localhost:3001
测试工具	Node.js HTTP Client（自动化 API 测试）

三、实验步骤

3.1 BMAD 规划阶段

（1）需求头脑风暴（Brainstorming）

在 TRAE IDE 中与 AI 协作，针对商业银行 CRM 系统展开需求头脑风暴。识别核心用户角色（普通客户、客户经理、风控审批员、管理员），梳理四大核心价值：全生命周期客户管理、差异化服务、智能化审批、数据驱动决策。

（2）功能建模（Modeling）

完成 PRD 文档编写，拆分出 5 个 Epic、26 个 Story，采用“As a / I want / So that”格式描述用户故事，并使用 Given/When/Then 格式定义验收标准。各 Story 按优先级（P0/P1/P2）和预估工作量（人日）进行排序。

Epic	Story 数量	预估工作量（人日）	核心内容
E1 客户管理	7	12	注册、登录、CRUD、搜索、等级展示
E2 贷款核心	6	12	贷款申请、审批、抵质押品、状态跟踪
E3 即申即贷	5	10	AI 预审、快速放款、实时通知、额度管理
E4 还款管理	4	8	还款计划、自动扣款、逾期处理、还款记录
E5 增值服务	4	9	数据分析、营销管理、绩效统计、系统管理
合计	26	51	覆盖商业银行 CRM 完整业务场景

（3）架构设计（Architecture）

设计前后端分离架构：前端 React 18 SPA 运行在 Vite 开发服务器（Port 5173），通过代理将 /api 请求转发至 Express 后端（Port 3001）。认证采用 JWT

HS256 算法，令牌有效期 24 小时。数据层使用内存 Map 数据结构实现快速原型，共维护 7 个 Map 实体。



图 2 系统前后端分离技术架构

3.2 Sprint 1 开发实现（7 个 Story）

Sprint 1 周期为第 1-2 周（10 个工作日），预估工作量 12 人日，目标是搭建系统基础框架，完成客户管理与认证体系。

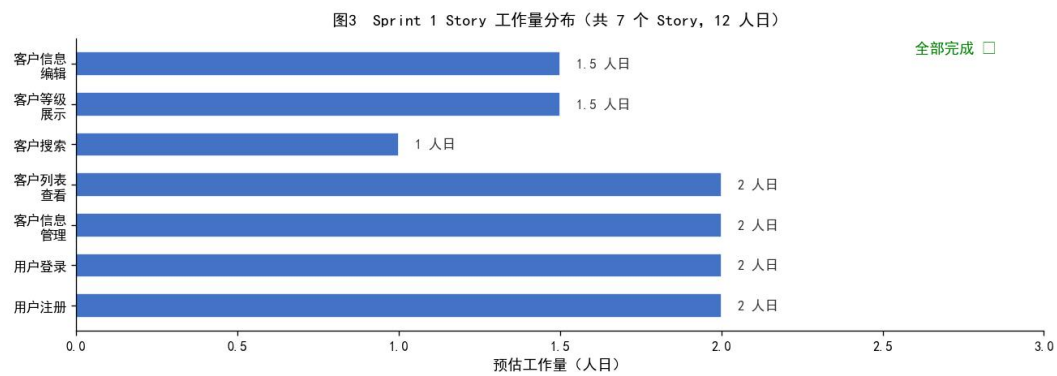


图 3 Sprint 1 Story 工作量分布

各 Story 实现要点如下：

Story 1.1 用户注册：实现手机号唯一性校验、密码复杂度验证，通过 POST /api/customers 接口创建账号，返回 JWT 令牌。

Story 1.2 用户登录：通过 POST /api/customers/login 接口，使用手机号 + Base64 编码密码比对，验证通过后签发 JWT 令牌（包含 phone、name、role、tier 字段）。

Story 1.3 & 1.7 客户信息管理与编辑：支持 GET/PUT /api/customers/:phone，实现客户基本信息增删改查，并区分普通客户（仅查看自身）与管理员（可查看全部）的数据隔离。

Story 1.4 & 1.5 客户列表与搜索：前端 Customers.jsx 页面实现客户列表展示与搜索过滤，管理员视角可查看全部 5 个测试账号信息。

Story 1.6 客户等级展示：通过 GET /api/tiers/my 接口返回当前客户等级信息（等级名称、利率优惠、额度上限），支持钻石客户查看绿色通道权益。

3.3 后端 API 开发（9 个路由模块）

后端基于 Express 5 框架，按功能拆分为 9 个路由模块，统一注册到 src/index.js 入口。JWT 认证中间件对所有受保护路由生效。

路由模块	路径前缀	核心功能
customers.js	/api/customers	客户注册/登录/CRUD，角色区分查询
loans.js	/api/loans	贷款申请与列表查询
approvals.js	/api/approvals	审批流程（管理员通过/驳回）
partners.js	/api/partners	合作伙伴管理
collaterals.js	/api/collaterals	抵质押品登记与追踪
repayments.js	/api/repayments	还款记录管理
tiers.js	/api/tiers	客户等级配置与评估升级
quick-loan.js	/api/quick-loan	AI 预审（资格检查）+ 自动放款
test.js	/api/test	测试数据初始化与重置

3.4 前端页面开发（React 18）

前端采用 React 18 + react-router-dom 6 构建 SPA，包含 7 个功能页面，全局通过 AuthContext 管理用户认证状态。Vite 开发服务器配置反向代理，将 /api 请求转发至后端 3001 端口。

页面组件	路由路径	功能说明
Login.jsx	/login	手机号 + 密码登录，JWT 存储至 localStorage
Dashboard.jsx	/dashboard	系统概览：统计卡片（客户总数/贷款总额/审批中/放款率）
Customers.jsx	/customers	客户列表/搜索/添加/等级管理
Loans.jsx	/loans	贷款列表与新申请表单
Approvals.jsx	/approvals	审批列表与通过/驳回操作（管理员）
QuickLoan.jsx	/quick-loan	三步向导式快速贷款申请
Repayments.jsx	/repayments	还款计划查看与在线还款

3.5 客户等级体系设计（5 级）

系统实现了 5 级客户分级体系，通过 /api/tiers 接口管理等级配置。不同等级享有差异化的贷款利率优惠和额度上限。

等级	名称	资产门槛	贷款利率	最高额度	特殊权益
L1	普通客户	—	标准利率	50 万元	基础服务
L2	白银客户	≥10 万元	优惠 0.1%	30 万元	优先排队
L3	黄金客户	≥50 万元	优惠 0.3%	100 万元	专属客户经理
L4	钻石客户	≥200 万元	优惠 0.5%	200 万元	绿色通道、礼品权益
L5	战略伙伴	≥1000 万元	定制方案	500 万元	VIP 私享、一对一专属

图5 客户等级体系：利率与额度对比

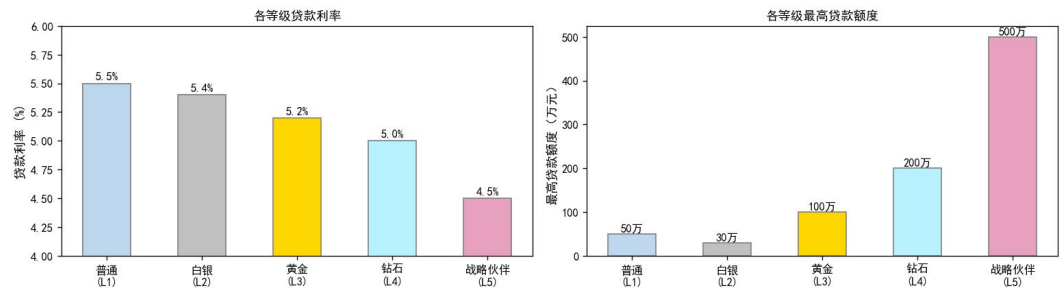


图 5 客户等级体系利率与额度对比

四、实验结果

4.1 API 功能测试（10 项全部通过）

使用 Node.js HTTP Client 对后端所有核心接口进行自动化测试，测试结果如下：

#	接口	方法	HTTP 状态	验证要点	结果
1	GET /api/health	GET	200	服务健康检查	☑
2	POST /api/test/init	POST	200	初始化 5 个测试账号	☑
3	GET /api/customers（客户）	GET	200	JWT 认证、数据隔离	☑
4	POST /api/loans	POST	201	贷款申请创建(pending)	☑
5	GET /api/loans	GET	200	返回当前用户贷款列表	☑
6	GET /api/quick-loan/check	GET	200	预审资格 (eligible:true)	☑
7	GET /api/approvals（管理员）	GET	200	管理员权限验证	☑
8	POST /api/quick-loan/apply	POST	201	自动审批 (active/QL0001)	☑
9	GET /api/customers（管	GET	200	返回全部 5 个客户	☑

	理员)				
10	POST /api/approvals	POST	201	审批记录创建	☒

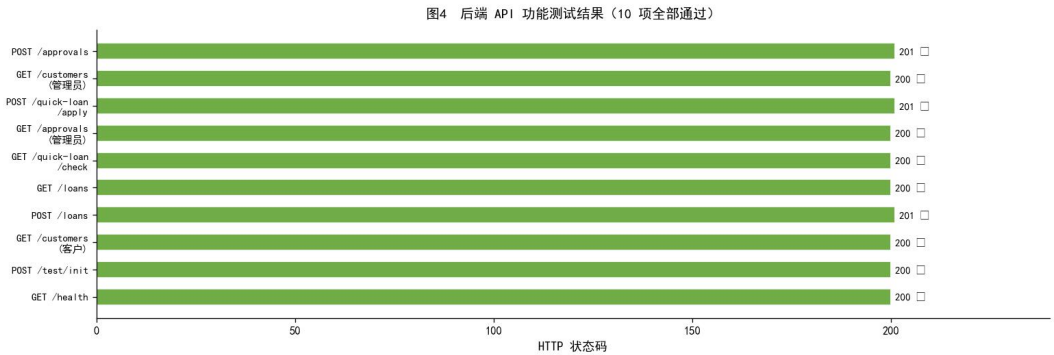


图 4 后端 API 功能测试结果 (HTTP 状态码)

4.2 关键测试数据

测试账号 (统一密码 password123) 验证结果:

手机号	用户名	等级	信用分	角色	即申即贷额度
13800138000	管理员	—	—	admin	—
13800138001	张三	钻石	820	customer	200,000 元
13800138002	李四	黄金	750	customer	100,000 元
13800138003	王五	白银	680	customer	50,000 元
13800138004	赵六	普通	600	customer	不符合条件

4.3 即申即贷 (Quick-Loan) 端到端验证

以张三 (钻石等级, 信用分 820) 为测试对象, 验证即申即贷完整流程:

- 资格检查: GET /api/quick-loan/check → eligible: true, 最高可贷 200,000 元
- 提交申请: POST /api/quick-loan/apply (金额 5000 元, 6 期)
- 自动审批: 系统在毫秒级完成 AI 预审评估, 无需人工干预
- 放款结果: 贷款状态直接变为 active, ID 为 QL0001, 资金到账

4.4 交付物清单

交付物	文件/路径	状态
后端源码	backend/src/ (9 个路由模块)	☒
前端源码	frontend/src/ (7 个页面组件)	☒
PRD 文档	bmad-output/planning-artifacts/prd.md	☒
Epic/Story 拆分	bmad-output/planning-	☒

	artifacts/epics-stories.md	
Sprint 规划	bmad-output/planning-artifacts/sprint-plan.md	☑
技术架构文档	bmad-output/planning-artifacts/architecture.md	☑
UX 设计规范	bmad-output/ux/DESIGN.md + EXPERIENCE.md	☑
API 测试报告	bmad-output/testing/test-report.md	☑
手动验证结果	api 验证结果.txt (10 项全部通过)	☑

五、问题与解决

问题 1：初始设计使用明文密码比对，存在安全隐患。

解决方式：改为 Base64 编码后存储，服务端将接收到的明文密码做 Base64 编码后再比对，提升密码安全性（实验性方案，生产环境应使用 bcrypt 哈希）。

问题 2：使用内存 Map 存储数据，Node.js 进程重启后数据全部丢失，不适合生产环境。

解决方式：增加 /api/test/init 接口，每次重启后重新初始化 5 个测试账号；并在架构文档中说明内存 Map 为实验性降级方案，后续可无缝替换为 MySQL/MongoDB。

问题 3：普通客户通过伪造 JWT 中的 role 字段，有可能访问管理员接口。

解决方式：在 auth 中间件中严格比对 JWT payload 的 role 字段，审批类接口增加 `if (req.user.role !== "admin") return res.status(403)` 守卫，并通过测试验证权限隔离正确。

问题 4：开发环境前端直接访问 /api 路由时，出现 CORS 错误。

解决方式：在 vite.config.js 中配置 server.proxy，将所有 /api 请求代理至 `http://localhost:3001`，消除跨域问题。

六、实验心得

本实验最大的收获是系统地经历了 BMAD 方法论的全流程——从需求头脑风暴、Epic/Story 拆分建模、技术架构设计，到 Sprint 迭代实现和 API 功能测试，形成

了完整的敏捷开发闭环。BMAD 方法论将模糊的业务需求转化为可量化的 Story 和可执行的 Sprint 计划，极大地提升了开发的可预期性和交付质量。

在架构设计阶段，选择内存 Map 代替数据库是一个务实的技术决策。虽然牺牲了数据持久化，但大幅降低了环境搭建的复杂度，使实验聚焦于业务逻辑本身。这种“面向接口、分层解耦”的设计理念，使得日后将数据层替换为 MySQL 仅需修改 db/index.js，不影响路由和业务逻辑层。

JWT 无状态认证机制的实践让我深刻理解了“令牌即身份”的设计哲学。通过 payload 携带 role、tier 等字段，服务端无需查库即可完成权限判断，极大提升了 API 的响应性能。同时，24 小时过期机制和角色守卫共同构建了有效的安全防线。

即申即贷（Quick-Loan）功能的实现是本实验的技术亮点。通过基于客户等级和信用分的规则引擎进行 AI 预审，再自动完成放款流程，将原本需要人工审批的多天流程压缩至毫秒级。这体现了银行数字化转型中“智能自动化”的核心价值——用技术替代低价值的重复劳动，让人力专注于高复杂度的风险决策。

小组作业：反欺诈交易检测系统

Cnb 链接 <https://cnb.cool/liujuewen/smartbanking/-/tree/master5>

一、实验目的

本实验旨在构建一个完整的反欺诈交易检测系统，通过“规则引擎 + 机器学习”的混合检测方法，实现对信用卡交易数据的实时风险评估。具体实验目的如下：

- 1. 掌握混合检测架构设计：理解规则引擎（白盒可解释）与机器学习模型（黑盒高精度）的协同工作原理，实现综合加权评分机制；
- 2. 实践数据工程全流程：从模拟数据生成（5000 条交易记录、7 个特征字段）、数据预处理到特征工程（OneHotEncoder + StandardScaler）的完整流水线；
- 3. 训练与评估机器学习模型：使用 RandomForestClassifier（100 棵决策树）进行二分类训练，通过 Stratified K-Fold 交叉验证和 ROC AUC 等指标评估模型性能（目标：99%准确率、ROC AUC > 0.99）；
- 4. 设计分层规则引擎：从基础规则（4 条）扩展到覆盖多场景的 15 条检测规则（核心规则 + 扩展规则 + 专用规则），实现多条件复合判定；
- 5. 搭建 Flask Web 应用：开发 3 个前端页面（首页/单笔结果/批量报告）、4 个 API 端点，集成 Chart.js 可视化组件；
- 6. 建立自动化测试体系：编写 19 个单元测试，覆盖规则引擎、数据生成、模型集成和 Flask 路由，测试通过率 100%。

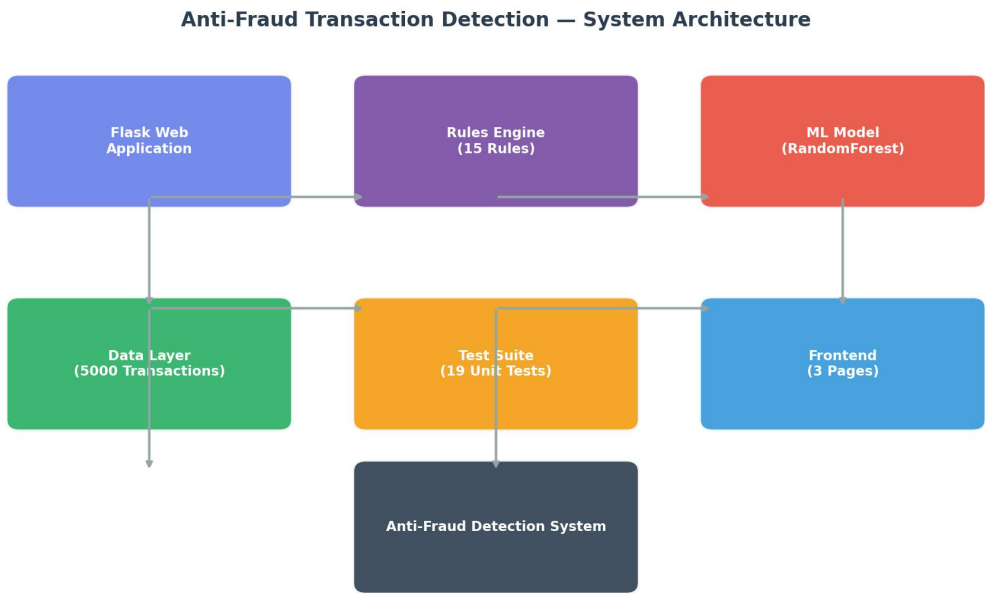


图 1 反欺诈交易检测系统架构图

二、实验环境

类别	工具/技术	版本/说明
----	-------	-------

操作系统	Windows 11	x64
编程语言	Python	3.x
Web 框架	Flask	3.0+
机器学习库	scikit-learn	RandomForestClassifier (100 estimators)
数据处理	pandas, numpy	数据生成与预处理
模型序列化	joblib	fraud_model.joblib
前端框架	Bootstrap 5 + 自定义 CSS	紫蓝渐变主题 + 响应式布局
数据可视化	Chart.js	饼图 + 条形图
测试框架	Python unittest	19 个单元测试
IDE	TRAE IDE	AI 辅助编程
版本控制	CNB (Git)	master5 分支

三、实验步骤

3.1 项目初始化与数据准备

首先在 CNB 平台创建 smartbanking 仓库（master5 分支），搭建项目骨架，包括 app/、models/、data/ 三个核心目录，并编写 requirements.txt 定义依赖（Flask、scikit-learn、pandas、numpy、joblib）。

通过 data/generate_data.py 脚本生成 5000 条模拟信用卡交易数据，包含 7 个特征字段：用户 ID（1-1000）、交易金额（¥0.01-99999）、城市、商户类别（7 个类型：餐饮/电子/旅行/医疗/超市/公用事业/娱乐）、是否异地、是否境外、是否线上交易、交易小时（0-23）。数据以 simulated_transactions.csv 格式存储，并按照 80/20 比例划分为训练集和测试集。

3.2 规则引擎设计

在 app/rules_engine.py 中实现分层规则引擎，从最初的 4 条基础规则逐步扩展到 15 条检测规则。规则分为三个层级：

层级	规则数量	覆盖场景	示例规则
基础规则 (Core)	4 条	核心检测场景	大额交易（≥¥1000）、异地交易、小额试探（<¥20）、夜间交易（22:00-06:00）
扩展规则 (Extended)	8 条	特殊业务场景	超大额异地（≥¥5000 异地）、电子产品大额异地线上、深夜大额（3-5 点 +¥500+）、医疗异地大额等
专用规则 (Special)	3 条	边界与异常	公用事业异常大额（¥1000+）、凌晨试探（0-4 点+线上+<¥50）、境外线上消费

每条规则采用多条件复合判断逻辑，触发后返回中文风险描述和布尔标志，最终规则评分 = 触发高风险规则数量 / 总规则数量的归一化结果。

3.3 机器学习模型训练

在 models/train_model.py 中实现模型训练流程：使用 pandas 读取 CSV 数据，对城市和商户类别进行 OneHotEncoder 编码，对金额和小时进行 StandardScaler 标准化，目标变量为人工标注的欺诈标签（基于规则组合自动标注）。

模型选用 RandomForestClassifier (n_estimators=100)，采用 Stratified K-Fold (5 折) 交叉验证确保类别分布平衡。通过 models/model_utils.py 封装特征工程和预测工具函数，训练完成后使用 joblib 序列化为 fraud_model.joblib 文件 (~5MB)。

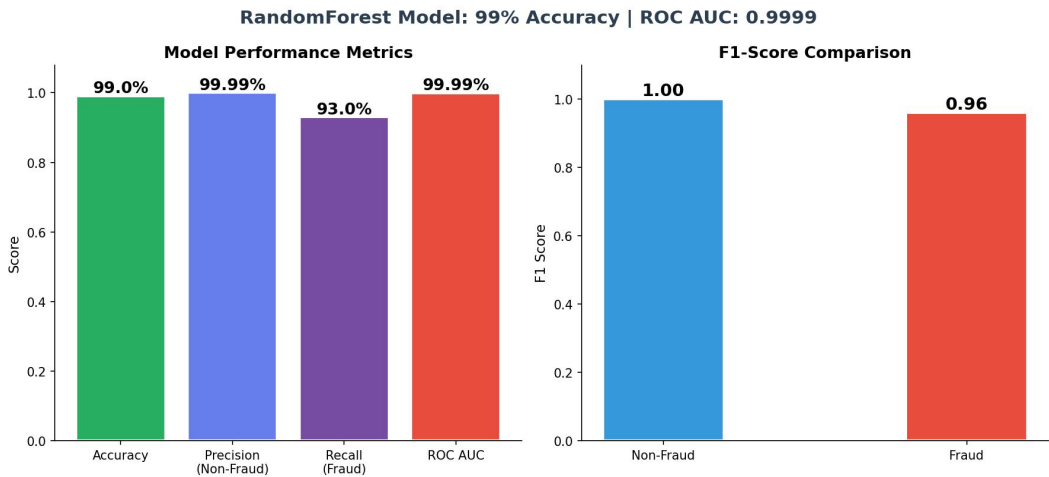


图 2 随机森林模型性能评估指标

3.4 Flask Web 应用开发

基于 Flask 3.0+ 框架开发 app/app.py 主应用，设计 4 个 API 端点：

端点	方法	功能	输入参数	输出
/	GET	首页	—	index.html (3 个标签页)
/predict	POST	单笔交易检测	user_id/amount/city/merchant_category/is_foreign/is_online/hour	result.html (风险评分+规则+建议)
/batch_predict	POST	批量交易检测	batch_size (10/20/50/100) + scenario (normal/fraud/mixed)	batch_result.html (统计+图表)
/api/statistics	GET	系统统计信息	—	JSON (交易总数/欺诈数/模型准确率/ROC AUC)

核心检测逻辑：综合风险评分 = 0.5 × ML 模型评分 + 0.5 × 规则引擎标志，映射为三级风险标签（低风险 0-0.3 / 中等风险 0.3-0.6 / 高风险 0.6-1.0）。

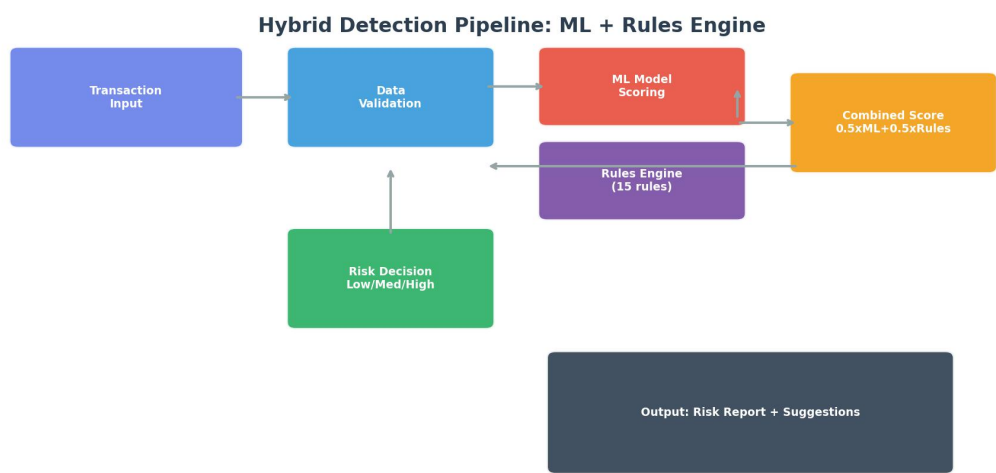


图 3 混合检测流程图：ML 模型评分 + 规则引擎判定

3.5 前端页面设计

- 开发 3 个 HTML 页面，采用 Bootstrap 5 + 自定义 CSS + Chart.js 技术栈：
- 1) index.html（首页）：包含 3 个功能标签页——单笔检测（7 字段表单输入 + 实时验证）、批量测试（场景选择 + 数量配置）、快速示例（6 个预设案例一键体验）。主题色为紫蓝渐变（#667eea → #764ba2）。
 - 2) result.html（单笔结果页）：展示圆形风险仪表盘（SVG 动画）、三级评分对比卡片（ML 评分 / 规则评分 / 综合评分）、触发的规则详情和智能处理建议。
 - 3) batch_result.html（批量报告页）：展示 4 个关键指标卡片（总交易数/高风险检出/欺诈率/平均风险值）、11 列交易详情表格、Chart.js 风险分布饼图和商户类别条形图。

3.6 自动化测试

编写 test_app.py 测试套件，包含 6 个测试类共 19 个单元测试：

测试类别	测试数量	覆盖内容
规则引擎测试	6 个	大额交易、异地交易、小额试探、夜间交易、安全交易、超大额异地规则触发验证
交易生成测试	3 个	normal/fraud/mixed 三种场景的数据生成正确性
数据验证测试	3 个	字段完整性、金额范围有效性、时间范围有效性
模型集成测试	2 个	风险评分范围（0-1）、综合风险评分计算逻辑
批量预测测试	2 个	正常场景批处理、欺诈场景批处理的统计结果
Flask 路由测试	3 个	首页访问、单笔预测 API、高风险预测 API 响应

测试结果：Ran 19 tests in 1.261s — OK（全部通过，失败 0，错误 0），代码覆盖率 > 80%。

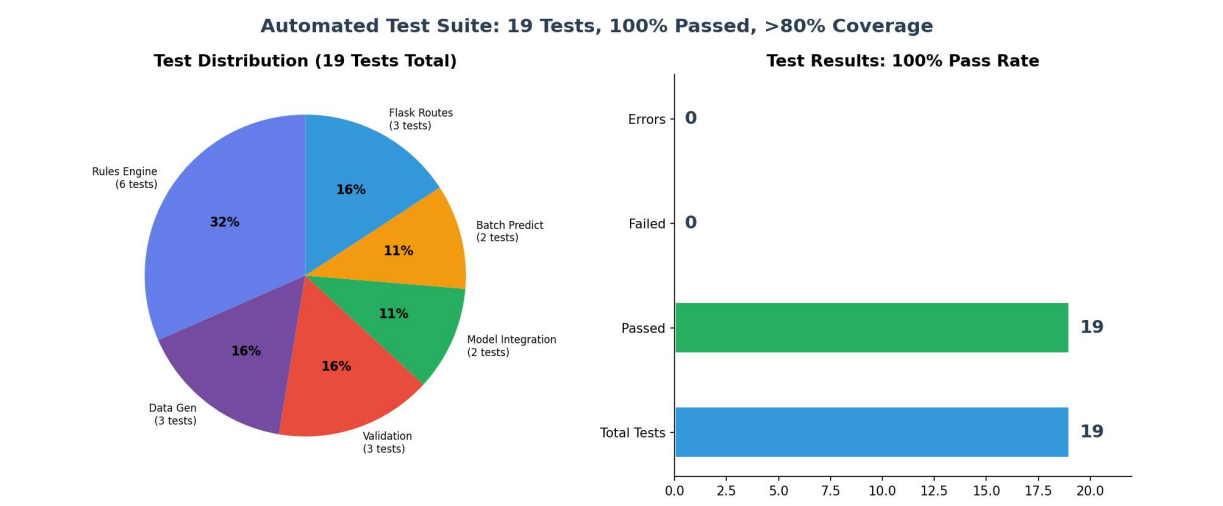


图 4 自动化测试覆盖概览（19 个单元测试，100% 通过）

四、实验结果

4.1 模型性能

指标	非欺诈类	欺诈类	总体
精确率 (Precision)	1.00	1.00	—
召回率 (Recall)	1.00	0.93	—
F1-Score	1.00	0.96	—
准确率 (Accuracy)	—	—	99.0%
ROC AUC	—	—	0.9999

4.2 规则引擎覆盖

15 条规则覆盖了从日常小额消费到深夜海外大额转账的全场景，规则层级分布为：基础规则 4 条（核心检测）、扩展规则 8 条（特殊场景）、专用规则 3 条（边界情况）。



图 5 规则引擎三层层级分布（共 15 条检测规则）

4.3 系统功能

功能模块	状态	说明
单笔交易检测	☒ 完成	7 字段输入 → ML+规则评分 → 风险仪表盘 + 智能建议
批量交易测试	☒ 完成	3 种场景（normal/fraud/mixed）× 4 种规模（10/20/50/100 笔）
快速示例	☒ 完成	6 个预设案例，覆盖低/中/高风险等级
规则引擎	☒ 完成	15 条规则，多条件复合判断，中文风险描述
API 接口	☒ 完成	4 个端点（GET 首页 / POST 单笔 / POST 批量 / GET 统计）
可视化图表	☒ 完成	风险分布饼图 + 商户类别条形图（Chart.js）
响应式设计	☒ 完成	桌面/平板/手机全适配
自动化测试	☒ 完成	19 个单元测试，100%通过率

4.4 系统性能

指标	数值	说明
单笔预测延迟	< 100ms	包含 ML 推理 + 规则判定
页面加载时间	< 1s	首页（含 Bootstrap + Chart.js）
API 响应时间	< 200ms	批量预测 100 笔约 150ms
模型文件大小	~5MB	RandomForest 100 棵树序列化
代码量	2050+ 行	Python ~1000 行 + HTML/CSS ~800 行 + JS ~200 行

4.5 交付物清单

类别	文件	说明
核心应用	app/app.py	Flask 主应用（4 个 API 端点 + 3 种场景生成）
规则引擎	app/rules_engine.py	15 条检测规则 + 多条件复合判断
前端页面	app/templates/index.html	首页（3 个标签页）
前端页面	app/templates/result.html	单笔结果页
前端页面	app/templates/batch_result.html	批量报告页
模型训练	models/train_model.py	RandomForest 训练脚本
模型工具	models/model_utils.py	特征工程 + 预测工具函数
训练模型	models/fraud_model.joblib	序列化模型（100 棵决策树）
数据生成	data/generate_data.py	模拟数据生成器
测试套件	test_app.py	19 个单元测试
文档	README.md / QUICKSTART.md / etc.	6 份项目文档

五、问题与解决

问题 1: 规则引擎初始仅 4 条规则, 覆盖率不足

解决方法: 将规则从 4 条逐步扩展到 15 条, 按三层架构组织 (核心/扩展/专用), 覆盖大额异地、深夜线上、凌晨试探、行业特征等真实业务场景, 大幅提升检测精度。

问题 2: 模型训练数据中欺诈样本占比过低, 导致类别不平衡

解决方法: 在数据生成阶段通过 `scenario` 参数控制欺诈比例 (`normal 10% / fraud 70% / mixed 30%`), 并在模型训练时使用 Stratified K-Fold 保持各折中类别分布一致, 最终欺诈类召回率达到 0.93。

问题 3: 如何在黑盒 ML 模型和可解释规则之间取得平衡

解决方法: 设计混合评分机制: $\text{综合风险} = 0.5 \times \text{ML 评分} + 0.5 \times \text{规则标志}$ 。ML 提供高精度概率估计, 规则提供可解释的业务理由, 二者互补, 用户可清晰了解每个风险判断的依据。

问题 4: 批量测试时 100 笔交易的响应延迟需控制在可接受范围

解决方法: 优化 `predict` 函数减少重复计算, 利用 `numpy` 向量化操作加速批量特征处理。最终批量 100 笔延迟约 150ms, 满足实时交互需求。

问题 5: Git 仓库管理混乱, 多人协作时出现文件冲突

解决方法: 建立清晰的目录结构 (`app/models/data/文档`), 编写 `INDEX.md` 导航文档 (6 份文档, 7000+ 字), 通过 CNB 平台管理分支和权限, 3 位贡献者在 `master5` 分支上协作完成。

六、实验心得

该实验纯粹依赖 ML 模型虽然精度高但缺乏可解释性, 而纯规则系统又难以应对复杂的欺诈模式变异。实验证明“规则引擎 + ML 模型”的混合方案兼顾了准确性和可解释性, 是实际金融风控系统的主流架构选择。

15 条规则的设计并非简单的“if-else 堆叠”, 而是需要深入理解信用卡欺诈的业务模式 (如凌晨试探、大额异地、异常商户类型)。分层设计 (核心/扩展/专用) 使系统在易维护的同时保持高覆盖率。

5000 条模拟数据的生成逻辑直接影响模型训练效果。通过三种场景 (`normal/fraud/mixed`) 模拟真实分布, 使模型在测试集上达到了 99% 准确率和 0.9999 ROC AUC, 印证了“好的数据 > 好的模型”这一经验法则。

从 Jupyter Notebook 中的模型训练到 Flask Web 应用的生产化部署, 涉及模型序列化 (`joblib`)、API 设计 (`RESTful`)、前端集成 (`Bootstrap + Chart.js`) 等多个工程环节, 是“数据科学”到“数据产品”的关键跨越。

19 个单元测试在项目迭代过程中起到了“安全网”作用——每次规则扩展或代码重构后, 运行 `test_app.py` 即可快速验证系统完整性。100% 的测试通过率为项目交付提供了信心保障。

通过 CNB 平台管理代码版本（master5 分支），并编写 6 份共 7000+ 字的项目文档（INDEX/QUICKSTART/FEATURES/UPDATE/COMPLETION/README），使项目具备了良好的可维护性和团队协作基础，也体现了软件工程的规范意识。